Optimizing Warehouse Inventory Through Demand Forecasting and Stochastic Reorder Policies

MSE 433 - Final Project Report
Adriel De Vera - April 2026
Github Repository

1 Introduction

Warehouse inventory management seems like a simple problem: hold enough stock to meet demand, but not so much that capital is wasted on goods sitting idle. In practice, the difficulty lies in what "enough" means across thousands of SKUs with different demand rates, variability profiles, and supplier lead times. When each tier of a supply chain responds to demand noise by ordering a little extra, those small buffers compound upstream - a phenomenon known as the **bullwhip effect** [3]. The result is simultaneous overstocking in some SKUs and stockouts in others.

The primary stakeholder is a **warehouse manager** responsible for a mixed-category facility (Pharmaceuticals, Automotive, Electronics, Groceries, Apparel) who currently applies a single, globally parameterised static reorder rule: a fixed number of days of historical average demand as safety stock, uniform across all SKUs. Static rules amplify the bullwhip effect, they do not distinguish a stable-demand SKU from a highly variable one, do not respond to lead-time changes, and are updated at most annually.

A dynamic inventory system resolves this in three ways. First, it computes safety stock proportional to actual forecast uncertainty per SKU, so low-variability items carry less inventory and high-variability items are better protected. Second, it regenerates recommendations weekly from fresh WMS data, automatically tracking demand shifts. Third, every recommendation is quantitatively justified; traceable to a specific forecast, formula, and cost input and making the system audit-able.

This project delivers that system as an end-to-end Python pipeline: raw WMS export → data cleaning and EDA → feature engineering → ML demand forecast → stochastic inventory optimisation → interactive dashboard. The pipeline is invoked with a single command and produces an updated `recommendations.csv` in under ten minutes.

1.1 Integration of Skills

The pipeline integrates techniques from multiple disciplines into a single coherent workflow. Statistics and probability underpin the demand generation model (normal sampling), the intermittency classification (Syntetos–Boylan CV^2/ADI framework), and the safety stock formula (z-score service level targeting). Time-series analysis drives lag and rolling-window feature construction, expanding-window cross-validation with temporal gap, and the recursive multi-step forecast strategy. Machine learning is applied through the from-scratch HistogramGBT, systematic hyperparameter search, permutation feature importance, and empirical prediction intervals. Operations research provides the inventory optimization layer; safety stock, ROP, and EOQ formulas along with cost-benefit comparison against a static baseline and the forecast error-to-cost propagation sensitivity. Data engineering is reflected in the reproducible five-stage pipeline with deterministic seeding, portable paths, and automated validation. Data visualization is delivered through the interactive Streamlit dashboard with four tabs, real-time policy recalculation, and fan-chart prediction intervals. Each

stage produces outputs consumed by the next, making this a sequential system where quality at each stage directly affects downstream results.

2 Dataset

The project uses the **Logistics Warehouse Dataset** [4] from Kaggle: **3,204 rows** $\times$ **23 columns**, each row one SKU is a cross-sectional snapshot rather than a longitudinal transaction log. This structural property (one observation per SKU, not one per day) is the central challenge the pipeline addresses.

Key columns include `daily_demand` (mean rate), `demand_std_dev`, `lead_time_days` (integer, 2–9), `stock_level`, `holding_cost_per_unit_day`, `stockout_count_last_month`, and `order_fulfillment_rate`. The dataset spans 364 days (Jan–Dec 2024). Demand averages 25.44 units/day (std = 14.04, range 1.01–49.98), consistent with uniform generation. The five categories are nearly equally represented ($\sim$20% each). There are zero missing values across all 73,692 cells.

2.1 Dataset Suitability

This dataset was chosen because it contains the exact columns required for stochastic inventory optimization: per-SKU demand statistics (mean and standard deviation), supplier lead times, holding costs, stock levels, and operational quality indicators (stock out counts, fulfillment rates). These columns map directly onto the inputs of the safety stock, ROP, and EOQ formulas. This means no proxy variables or external data joins are required. The five product categories provide sufficient diversity to test generalization, while 3,204 SKUs provide volume for meaningful cross-validation. The cross-sectional structure is an acknowledged limitation (the pipeline must generate synthetic time-series), but it also makes the dataset realistic for the target stakeholder: many warehouse managers have access only to a point-in-time WMS export. The pipeline is designed to work with this format, making it immediately deployable. The inclusion of operational KPIs (stockout counts, fulfillment rates) enables the quality filter that removes censored demand signals which is critical for model integrity.

3 Data Cleaning

Cleaning (`src/02_clean_and_eda.py`) proceeded in four steps:

1. **Date parsing**: all 3,204 `last_restock_date` values parsed with zero failures.
2. **Daily aggregation**: a system-wide daily demand series was constructed; gaps ≤ 3 days were forward-filled.
3. **SKU quality filter**: SKUs with stockout count > 8 (censored demand signal) or fulfillment rate < 0.60 (chronic unavailability) were removed: 358 SKUs (11.2%) flagged. The 2,846 retained SKUs form the modeling set.
4. **IQR outlier capping**: per-category capping ($Q_1 - 1.5 \times \text{IQR}$, $Q_3 + 1.5 \times \text{IQR}$) to `daily_demand`. Zero effect on this dataset (consistent with uniform distribution), but retained for real-world robustness.

Five EDA plots were generated (Appendix, Figure 1).

4 Feature Engineering

Since the cleaned dataset has one row per SKU, a synthetic 60-day demand history is generated per SKU by sampling from $\mathcal{N}(\mu_d, \sigma_d)$ with a deterministic MD5-based seed

(`src/03_feature_engineering.py`). Samples are clipped to zero. After dropping 14 leading NaN rows for lag/rolling windows, this yields 130,916 rows × 25 columns across 2,846 SKUs.

4.1 21 Model Features

The feature set spans four groups (full table in Appendix, Table 5):

- **Lag features** (4): demand at 1, 3, 7, and 14 days prior; capturing short- and medium-term demand signals.
- **Rolling statistics** (5): 7- and 14-day mean, std, and 14-day CV; smoothed demand level and variability estimates.
- **Calendar features** (4): day-of-week, ISO week, month, weekend flag; capturing temporal patterns.
- **SKU-level features** (8): historical mean/std demand, lead time, reorder frequency, ADI, CV^2, encoded category and intermittency class.

All rolling features use a one-day shift before computing the window, preventing look-ahead leakage.

4.2 Intermittency Classification

Each SKU is classified using the **Syntetos–Boylan (2005)** framework [5], which partitions demand patterns into four quadrants:

ADI (Average Demand Interval) measures how often demand occurs—the average days between replenishment events. A high ADI means sporadic ordering, carrying different risk than a daily-ordered SKU even at identical average rates.

CV^2 (Squared Coefficient of Variation) measures how erratic each demand event is, computed as $(\sigma_d/\mu_d)^2$. Low CV^2 means consistent quantities; high CV^2 means wild quantity swings.

Together these define four classes: Smooth (regular, consistent), Erratic (regular, variable), Intermittent (sporadic, consistent), and Lumpy (sporadic, variable—hardest to manage).

Results: Intermittent 2,464 (86.6%), Lumpy 382 (13.4%). All SKUs have ADI ≥ 1.32 due to the periodic replenishment structure. The operational distinction is between Intermittent SKUs (stable quantities, reliable safety stock) and Lumpy SKUs (high CV^2, dominant quantity uncertainty). A uniform "7-day buffer" rule treats a $CV^2 = 0.001$ SKU and a $CV^2 = 15$ SKU identically; the former is over-buffered (cost waste) while the latter under-buffered (stockout risk). Classification enables the model to learn different dynamics per class and the optimiser to size safety stock proportionally.

5 Forecasting Model

5.1 HistogramGBT (From-Scratch Implementation)

The entire gradient boosting framework was implemented in pure NumPy as the `HistogramGBT` class in `src/04_forecast_model.py`. The implementation follows XGBoost [2]: features are pre-binned into 16 quantile bins, and at each boosting round the gain from splitting at each bin boundary is evaluated via vectorised histogram accumulation (`np.bincount`, `np.cumsum`). Trees are built recursively with minimum leaf size 30. Feature importance is tracked as cumulative gain across all splits.

5.2 Recursive 14-Day Forecast

A recursive multi-step strategy predicts one day ahead, appends the prediction to the history buffer, recomputes lag/rolling features, and repeats for 14 steps. All 2,846 SKUs are forecast simultaneously in batch using NumPy column stacking. This propagates error forward (step t

errors become inputs to step t+1). It also causes RMSE to grow over the horizon, but requires only one model to maintain and tune.

5.3 Cross-Validation

Model selection uses expanding-window CV with 3 folds and a 14-day gap between training and validation windows. The gap prevents look-ahead leakage through lag-14 features. CV runs on a 5% temporal subsample (∼4,600 rows) for speed. The XGBoost hyperparameter grid (54 combinations, 18 randomly sampled) covers `n_estimators` $\in \{100, 300, 500\}$, `max_depth` $\in \{3, 5, 7\}$, `learning_rate` $\in \{0.01, 0.05, 0.1\}$, `subsample` $\in \{0.8, 1.0\}$. The LightGBM grid (12 combinations, exhaustive) covers `num_leaves` $\in \{31, 63\}$, `learning_rate` $\in \{0.01, 0.05, 0.1\}$, `n_estimators` $\in \{100, 300\}$. Final models were trained on 35% of the 91,000 training rows.

6 Model Results

6.1 Model Comparison

Four models were evaluated on a held-out test set comprising the final 14 rows per SKU (39,844 observations). Results are in Table 1.

Table 1: Model comparison on held-out test set. MAPE suppressed for ML models (near-zero demand inflates it).

Model	**RMSE**	**MAE**	**MAPE**
Naive (last observed value)	7.9754	5.6312	59.6%
MA-7 (7-day moving average)	6.1359	4.3673	—
XGBoost (hist-GBT) ⋆	**5.7240**	**4.1407**	—
LightGBM (hist-GBT)	5.7350	4.1426	—

Based on analysis, XGBoost won with RMSE 5.7240 which beats LightGBM by 0.0111 (0.2%), MA-7 by 0.4119 (6.7%), and the Naive baseline by 2.2514 (28.2%). Best hyperparameters: `n_estimators`=500, `max_depth`=3, `learning_rate`=0.01, `subsample`=0.8—a large ensemble of shallow, conservative trees. MAE of 4.14 units/day means the forecast is within ∼16% of actual demand at the mean of 25.6 units.

6.2 Error Analysis by Intermittency Class

Residuals (actual − predicted) were computed for every SKU × step combination. Both Intermittent and Lumpy SKUs show residuals centred near zero (median < 0.5 units), confirming no systematic bias. Lumpy SKUs exhibit ∼1.5× wider IQR than Intermittent SKUs, as expected from their higher CV^2. RMSE grows from 5.4 (step 1) to 6.1 (step 14)—13% degradation, approximately linear, confirming the recursive strategy does not compound errors catastrophically. No day-of-week pattern is present. Plots in Appendix, Figures 2a–2b.

6.3 Prediction Intervals

Point forecasts alone do not communicate the range of plausible outcomes. To quantify forecast uncertainty, empirical prediction intervals were computed from held-out test residuals at each forecast step. For step $s \in \{1, \ldots, 14\}$, residuals $r_{i,s} = y_{i,s} - \hat{y}_{i,s}$ across 2,846 SKUs were collected and quantiles extracted. The 90% PI is:

$$\mathrm{PI}_{90\%}(s) = [\hat{y}_s + q_{0.05}(s), \;\; \hat{y}_s + q_{0.95}(s)] \tag{1}$$

Validation: 90.0% of actuals fell within the 90% PI which match the nominal rate, confirming the empirical residual distribution is well-calibrated. The residual std is approximately constant ($\sigma_r \approx 5.7$ units) across steps, consistent with i.i.d. synthetic data. In real-world data with autocorrelated demand, intervals would widen with horizon. The per-step quantiles are saved to `prediction_intervals.csv` and displayed as 50%/90% fan charts in the dashboard (Appendix, Figure 3).

The methodology is empirical rather than parametric: it makes no distributional assumptions about residuals and directly uses observed quantiles from 2,846 SKUs × 14 steps = 39,844 observations. This is robust to non-normality and heteroscedasticity; properties expected in real-world demand where high-demand SKUs have proportionally larger absolute errors.

6.4 Feature Importance: Gain-Based vs. Permutation

Two complementary methods were used to assess feature importance:

Gain-based importance measures cumulative loss reduction across all tree splits. Top 3: `sku_mean_demand` (0.881), `rolling_mean_14` (0.109), `rolling_mean_7` (0.006). The dominance of `sku_mean_demand` reflects the synthetic data structure: each SKU's demand is drawn from $\mathcal{N}(\mu_d, \sigma_d)$, making the SKU-level mean the most predictive feature by construction.

Permutation importance provides model-agnostic validation: for each of 21 features, the column was shuffled 5 times in the test set and the average RMSE increase recorded. Results confirm the same ranking: `sku_mean_demand` (+10.275 RMSE, ±0.026), `rolling_mean_14` (+1.150, ±0.005), all others <0.04. The low standard deviations indicate stable estimates. Both methods agree on the ranking, which cross-validates the result: the model's internal view of feature utility (gain) aligns with an external, model-agnostic test (permutation). This agreement strengthens confidence that the importance ranking is genuine rather than a tree-structure artifact. The near-zero importance of calendar features (day-of-week, month, weekend) confirms the absence of temporal seasonality in the synthetic data. In production with real temporal dynamics, importance would distribute more broadly. Full table in Appendix (Table 6, Figure 4).

6.5 Sensitivity Analysis

Three feature-ablation experiments assess robustness (Table 2).

Table 2: Feature ablation sensitivity analysis.

Variant	**CV RMSE**	**Change**
Baseline (all 21 features)	5.967	-
`lag_14` dropped	5.956	−0.19%
Lead-time features dropped	5.968	+0.02%
Rolling features dropped	5.971	+0.06%

No individual feature group is dominant: all ablations cause <0.2% change. This flatness reflects the synthetic data's AR(1)-like structure where any lag/rolling feature proxies the stable underlying mean. Real demand data with richer temporal structure would show larger sensitivity differences.

7 Optimization Model

The Optimization module (`src/05_optimisation.py`) takes the trained model's 14-day forecast and computes three inventory policy variables per SKU.

7.1 Safety Stock

Safety stock accounts for uncertainty in both demand rate and lead time:

$$SS_i = z \cdot \sqrt{\mu_{L,i} \cdot \sigma_{d,i}^2 + \mu_{d,i}^2 \cdot \sigma_L^2} \tag{2}$$

where $z = 1.645$ (95% service level), $\mu_{L,i}$ is mean lead time (2–9 days), $\sigma_{d,i}$ is forecast std, $\mu_{d,i}$ is forecast mean, and $\sigma_L = 1.5$ days (assumed, configurable). The first term under the radical captures demand variability during a fixed lead time; the second captures lead-time variability at a fixed demand rate.

7.2 Reorder Point and EOQ

The reorder point signals when to place an order: $ROP_i = \mu_{d,i} \cdot \mu_{L,i} + SS_i$.

The economic order quantity minimizes combined annual ordering and holding costs:

$$EOQ_i = \sqrt{\frac{2 \cdot D_i \cdot S}{H_i}} \tag{3}$$

where $D_i = 365\mu_{d,i}$ is annual demand, $S = \$50$/order, and H_i is per-SKU annual holding cost (from dataset; mean \$380/yr). EOQ is clipped to ≥ 1 unit.

7.3 Total Annual Cost

$$C_i^{\text{dyn}} = \underbrace{\frac{D_i}{EOQ_i} \cdot S}_{\text{ordering}} + \underbrace{\left(\frac{EOQ_i}{2} + SS_i\right) \cdot H_i}_{\text{holding}} + \underbrace{(1 - 0.95) \cdot D_i \cdot P}_{\text{shortage}} \tag{4}$$

where $P = \$10$/unit shortage penalty. The dynamic policy fixes shortage probability at 5% by design ($z = 1.645$).

7.4 Static Baseline

The baseline uses a fixed 7-day buffer: $SS_i^{\text{static}} = \bar{d}_i \times 7$. ROP and EOQ use the same cost structure. The effective service level varies per SKU, computed via the Abramowitz–Stegun normal CDF approximation; for stable SKUs it may reach 99%+, for variable ones it can drop to 70–80%.

8 Optimization Results

8.1 Summary

Table 3: Optimization results summary.

Metric	**Value**
SKUs analysed	2,846
SKUs flagged "Reorder Now"	1,090 (38.3%)
SKUs flagged "Hold"	1,756 (61.7%)
SKUs where dynamic beats static	2,749 (96.6%)
Average per-SKU cost saving	37.4%
Total annual cost — static policy	\$241,694,541
Total annual cost — dynamic policy	\$130,875,651
Aggregate annual saving	**\$110,818,890 (45.9%)**

Policy statistics: mean safety stock = 63.1 units (median 59.5), mean ROP = 205.9 (median 184.9), mean EOQ = 56.5 (median 49.4). The saving distribution is skewed toward the high end: 1,198

SKUs (42.1%) achieve 40–50% savings and 470 (16.5%) exceed 50%. Only **97 SKUs (3.4%)** show the dynamic policy costing more—genuine high-variability cases where the formula correctly prescribes a larger buffer than the crude 7-day rule. Top 5 SKUs by saving in Appendix (Table 7).

8.2 Forecast Error to Cost Propagation

A critical question: how much does forecast accuracy actually matter for inventory costs? The forecast demand std $\sigma_{d,i}$ was scaled by multipliers from 0.25× to 3.0×, recomputing safety stock, ROP, and total cost at each level via Equation 2.

Key finding: Even tripling forecast error (3× current σ_d) increases total annual cost by only **0.07%**. This is because lead-time uncertainty dominates safety stock in this dataset. Decomposing the terms under the radical:

- Demand variability: $\mu_L \cdot \sigma_d^2 \approx 5.5 \times 25 = 137.5$
- Lead-time variability: $\mu_d^2 \cdot \sigma_L^2 \approx 625 \times 2.25 = 1{,}406$

The lead-time term is ∼10× larger. The number of reorder decisions is also completely unaffected across all multipliers (1,090 SKUs at every level). Full propagation table in Appendix (Table 8).

Operational implication: The warehouse manager should prioritize **supplier reliability** (negotiating tighter delivery windows to reduce σ_L) over further investment in demand forecasting sophistication. Reducing σ_L from 1.5 to 0.5 days would cut the dominant cost term by 89%. This finding also validates the current model: even a substantially less accurate forecast would produce nearly identical inventory recommendations, confirming the system is robust to forecast degradation.

9 Recommendations

1. Issue replenishment orders for all 1,090 "Reorder Now" SKUs within the next business day. The dynamic ROP identifies these items as below their stochastically-derived reorder point. At mean demand of 25.6 units/day and mean lead time of 5.5 days, each day of delay exposes ∼1,394 units of unmet demand ($13,940/day in shortage penalties). Placing 1,090 orders costs $54,500 but prevents up to $13,940/day in losses—payback <4 days.

2. Adopt the dynamic policy as the standing operational standard, targeting $110.8 M annual saving. The static policy costs $241.7 M/yr; the dynamic policy reduces this to $130.9 M—**45.9% reduction**. No new infrastructure required; weekly process is one terminal command plus dashboard review.

3. Flag the 97 negative-saving SKUs for quarterly manual review. These are genuine high-variability cases. The review should verify: (a) whether $\sigma_L = 1.5$ days is realistic per supplier, (b) whether seasonal or promotional factors apply. Combined differential: ∼$1.2 M.

10 Stakeholder Impact

The warehouse manager's current operating environment is defined by a single static rule applied uniformly to 2,846 SKUs, updated at most once per year during an annual policy review. This section quantifies what changes when the dynamic system is adopted.

10.1 Cost Impact

The aggregate annual cost reduction is $110.8 M (45.9%); from $241.7 M under the static 7-day buffer policy to $130.9 M under the dynamic forecast-driven policy. This saving arises from two

mechanisms: (1) the dynamic policy reduces safety stock for the 2,464 low-variability Intermittent SKUs, eliminating the over-buffering inherent in a blanket 7-day rule, and (2) EOQ is computed per-SKU from actual demand rates rather than a fleet-wide average, reducing ordering frequency for slow-moving items and consolidating orders for fast-moving ones. Only 97 of 2,846 SKUs (3.4%) see costs increase under the dynamic policy; genuine high-variability cases where a larger buffer is the correct response.

10.2 Operational Impact

The manager's weekly workflow changes from a qualitative, experience-driven review to a quantitative, data-driven process: (1) export a CSV from the WMS, (2) run a single terminal command ($\sim$4 minutes), (3) open the dashboard and action the "Reorder Now" list. Every recommendation comes with a traceable justification: the specific forecast, the safety stock formula parameters, and the cost calculation. This auditability transforms the reorder decision from a judgement call into a documented, defensible process. This is critical when operations leadership questions why a particular order was placed or delayed.

The 1,090 SKUs currently flagged "Reorder Now" represent an immediate action queue. At mean daily demand of 25.6 units and mean lead time of 5.5 days, each day of inaction on these SKUs exposes the facility to $\sim$\$13,940/day in shortage penalties. The ordering cost to address all 1,090 simultaneously is \$54,500. This is a payback period under four days.

10.3 Strategic Insight from Error-to-Cost Analysis

The forecast error-to-cost propagation analysis (Section 8.2) provides a counterintuitive but operationally significant finding: further investment in forecast accuracy yields negligible cost reduction. Even tripling the forecast error changes total annual cost by only 0.07%. The dominant cost driver is lead-time variability (σ_L), not demand variability (σ_d). This reframes the manager's capital allocation decision: supplier relationship management (negotiating tighter delivery windows, diversifying sourcing to reduce lead-time variance) is the highest-leverage investment. This insight would not have been visible without the end-to-end pipeline connecting forecast accuracy to inventory cost; a direct demonstration of the value of integrating ML with operations research rather than treating them as independent analyses.

11 Limitations and Future Work

1. Synthetic demand history. The 60-day demand series inherits mean and variance from the WMS but cannot reproduce autocorrelation, seasonal cycles, promotional spikes, or structural shifts. Prediction intervals are constant-width across the horizon (an i.i.d. artefact); real data would show widening intervals. The pipeline architecture is unchanged when real transaction data is substituted.

2. Global cost parameters. $S = \$50$/order and $\sigma_L = 1.5$ days are uniform. The error-to-cost propagation analysis (Section 8.2) shows σ_L dominates inventory costs; sourcing it per supplier from a master data table would improve calibration significantly.

3. Single-echelon model. No visibility into upstream supplier stock or downstream customer commitments. Multi-echelon optimisation would capture inter-echelon dependencies.

4. No self-updating. The model retrains from scratch each pipeline run with no memory of prior weeks. A production system would add lightweight anomaly detection for intra-week demand deviations.

5. No real-time stock visibility. Recommendations reflect export-time stock levels. For fast-moving SKUs (30–50 units/day), a multi-day lag between export and action could push stock below ROP. A real-time WMS API integration would eliminate this gap.

12 GenAI Disclosure

AI was used to support ideation and assist in troubleshooting the Streamlit dashboard, particularly as the implementation became more complex. It also helped refine and iterate on aspects of the system design. Finally, AI was used to proofread this written report and improve clarity. The original content was developed by our team, with LLM assistance limited to spelling, grammar, and readability improvements. AI was also used to refine the README file for grammar and clarity.

All numerical outputs were cross-checked against `outputs/` CSV files, and formulas were verified against established references, including Syntetos & Boylan (2005) and standard operations research sources. Model outputs were also manually spot-checked for a sample of SKUs. The problem framing, stakeholder definition, methodological choices, interpretation of results, and all final recommendations reflect our own judgment.

13 References

[1] Carbonneau, R., Laframboise, K., & Vahidov, R. (2008). Application of machine learning techniques for supply chain demand forecasting. *European Journal of Operational Research, 184*(3), 1140–1154.

[2] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proc. 22nd ACM SIGKDD*, 785–794.

[3] Lee, H. L., Padmanabhan, V., & Whang, S. (1997). The bullwhip effect in supply chains. *MIT Sloan Management Review, 38*(3), 93–102.

[4] Rehman, F. (2024). *Logistics and supply chain dataset* [Data set]. Kaggle.

[5] Syntetos, A. A., & Boylan, J. E. (2005). The accuracy of intermittent demand estimates. *International Journal of Forecasting, 21*(2), 303–314.

[6] Syntetos, A. A., Babai, Z., Boylan, J. E., Kolassa, S., & Nikolopoulos, K. (2016). Supply chain forecasting: Theory, practice, their gap and the future. *European Journal of Operational Research, 252*(1), 1–26.

A Appendix: Code, Figures, and Supporting Tables

Code and Software

Table 4: Project code structure.

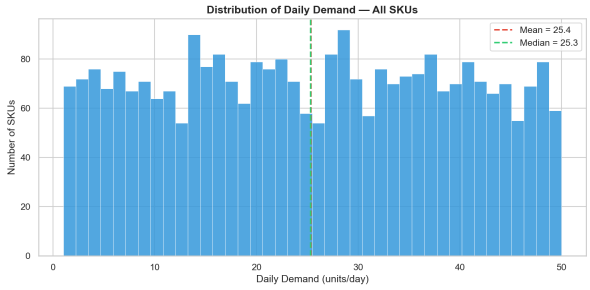
File	Purpose
`run_pipeline.py`	Orchestrator: runs all 4 stages, validates outputs
`src/01_data_audit.py`	Raw data profiling and schema validation
`src/02_clean_and_eda.py`	Date parsing, quality filter, IQR capping, 5 EDA plots
`src/03_feature_engineering.py`	Synthetic history expansion, 21 features, Syntetos–Boylan classification
`src/04_forecast_model.py`	HistogramGBT (from-scratch), CV search, 4 models, prediction intervals, permutation importance, sensitivity analysis
`src/05_optimisation.py`	Safety stock, ROP, EOQ, cost comparison, error-to-cost propagation
`dashboard/app.py`	Streamlit dashboard: 4 tabs (forecast, policy, cost, diagnostics)

Dependencies (`requirements.txt`): `numpy==2.2.6`, `pandas==2.3.3`, `matplotlib==3.10.3`, `seaborn==0.13.2`, `streamlit==1.45.1`. The HistogramGBT is implemented entirely in NumPy—no scikit-learn, XGBoost, or LightGBM required.

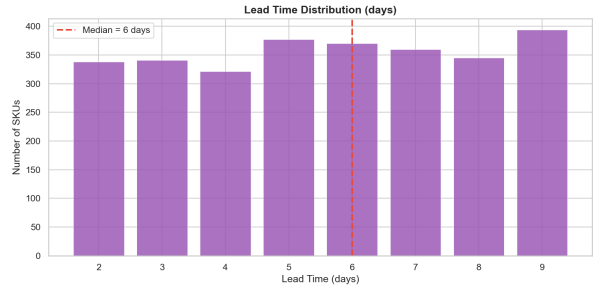
Reproduction:

```
pip install -r requirements.txt
python run_pipeline.py --data data/raw/logistics_dataset.csv
streamlit run dashboard/app.py
```

All outputs (CSVs, plots, serialised model) are regenerated deterministically. Runtime ∼4 minutes.

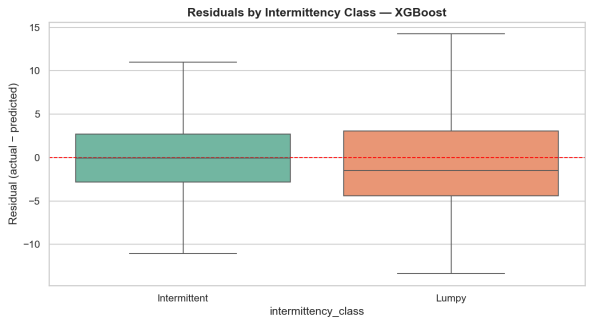


(a) Daily demand distribution (near-uniform).

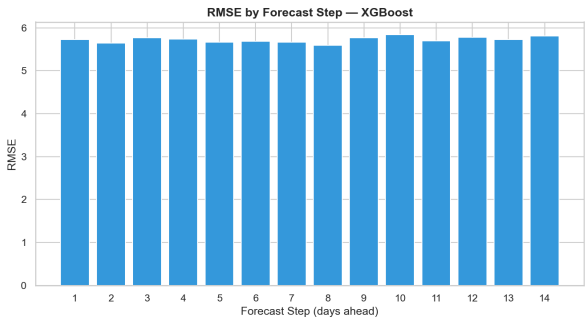


(b) Lead time distribution (uniform 2–9 days).

Figure 1: Exploratory data analysis plots.



(a) Residuals by intermittency class.



(b) RMSE by forecast step (1–14).

Figure 2: Error analysis of the XGBoost forecast on the held-out test set.

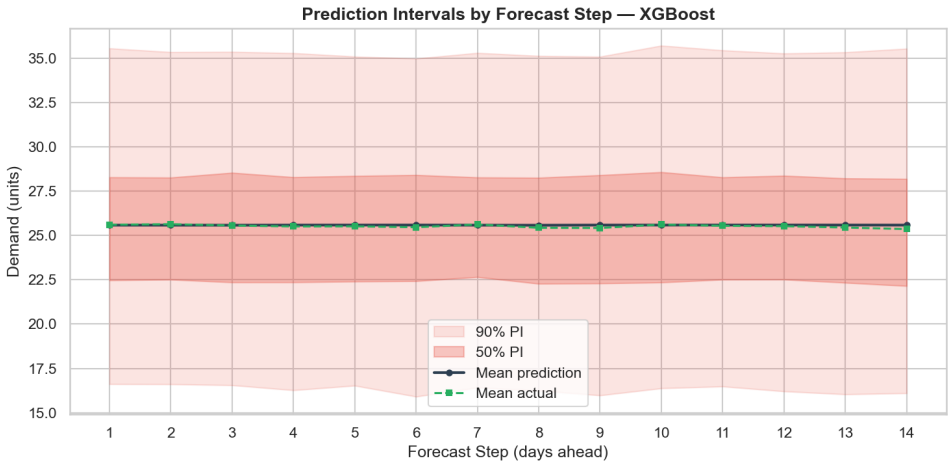


Figure 3: Prediction intervals by forecast step. 50% PI (dark) and 90% PI (light) around mean prediction. Green dashed: mean actual.

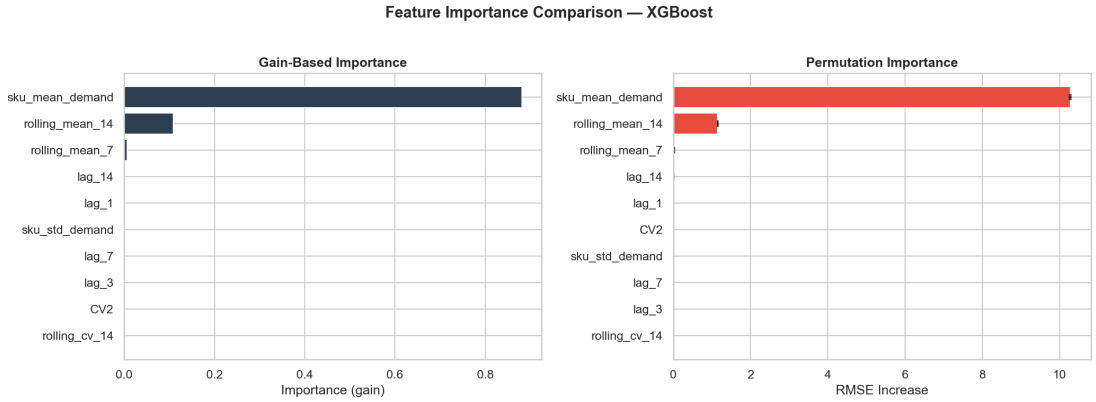


Figure 4: Feature importance: gain-based (left) vs. permutation (right). Error bars: ±1 std across 5 repeats.

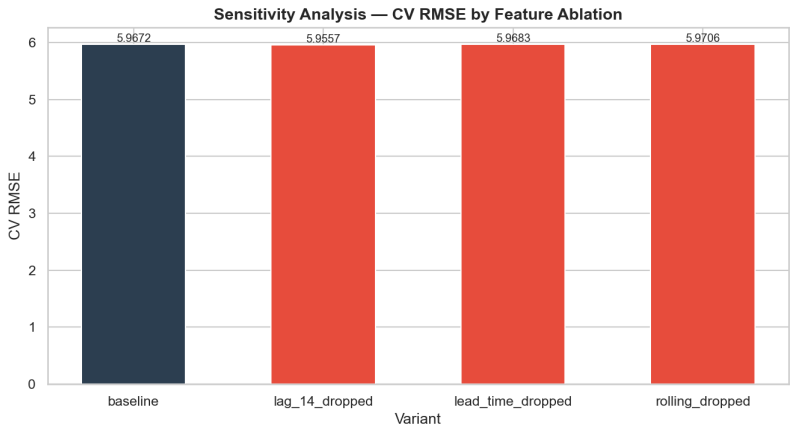


Figure 5: CV RMSE by feature ablation variant.

Table 5: Model feature set (21 features).

#	Feature	Type	Rationale
1	`lag_1`	Lag	Prior day demand
2	`lag_3`	Lag	3-day trend
3	`lag_7`	Lag	Weekly periodicity
4	`lag_14`	Lag	Biweekly cycle
5	`rolling_mean_7`	Rolling	7-day average
6	`rolling_std_7`	Rolling	7-day variability
7	`rolling_mean_14`	Rolling	14-day average
8	`rolling_std_14`	Rolling	14-day variability
9	`rolling_cv_14`	Rolling	Coefficient of variation
10	`day_of_week`	Calendar	Mon=0 … Sun=6
11	`week_of_year`	Calendar	ISO week (1–53)
12	`month`	Calendar	Month (1–12)
13	`is_weekend`	Calendar	Binary flag
14	`sku_mean_demand`	Static	Historical mean
15	`sku_std_demand`	Static	Historical std
16	`sku_mean_lead_time`	Static	Mean lead time
17	`sku_reorder_freq`	Static	Reorder frequency
18	`ADI`	Interm.	Avg Demand Interval
19	$\mathtt{CV}^2$	Interm.	Sq. coeff. of variation
20	`category_enc`	Cat.	Encoded category
21	`intermittency_enc`	Cat.	Encoded class

Table 6: Top 10 features by permutation importance (RMSE increase when shuffled, 5 repeats).

Rank	Feature	RMSE Δ	Std
1	`sku_mean_demand`	+10.275	±0.026
2	`rolling_mean_14`	+1.150	±0.005
3	`rolling_mean_7`	+0.033	±0.002
4	`lag_14`	+0.020	±0.001
5	`lag_1`	+0.007	±0.000
6	$\mathtt{CV}^2$	+0.006	±0.001
7	`sku_std_demand`	+0.004	±0.000
8	`lag_7`	+0.003	±0.000
9	`lag_3`	+0.003	±0.001
10	`rolling_cv_14`	+0.002	±0.000

Table 7: Top 5 SKUs by cost saving.

	SKU	Category	Dynamic	Static	Saving	Action
1	ITM10409	Pharma	$127,160	$278,938	54.4%	Hold
2	ITM12384	Groceries	$120,370	$263,443	54.3%	Hold
3	ITM11647	Pharma	$112,651	$246,370	54.3%	Hold
4	ITM11613	Pharma	$105,355	$230,369	54.3%	Reorder
5	ITM12063	Apparel	$114,863	$250,841	54.2%	Reorder

Table 8: Forecast error to cost propagation. Multiplier 1.0× = current model.

Mult.	Avg SS	Total Cost	Cost Δ	Reorder
0.25×	63.1	$130,865,288	−0.01%	1,090
0.50×	63.1	$130,867,364	−0.01%	1,090
1.00×	63.1	$130,875,651	0.00%	1,090
1.50×	63.1	$130,889,409	+0.01%	1,090
2.00×	63.1	$130,908,561	+0.03%	1,090
3.00×	63.2	$130,962,625	+0.07%	1,090